

MediKiosk — Prompt Engineering Task Sheet

Ayurveda module (Prakriti + Vikriti) and Allopathy module (SOCRATES) — for the person implementing prompt engineering + integration

What you're building, in one line

For every patient interaction, our system asks questions through an LLM. Each LLM call is built from three parts: (1) instructions we write telling it how to behave, (2) what the patient has said so far, and (3) reference material we retrieve to ground its questions. You are writing part (1) for two different domains — Ayurveda and Allopathy — and wiring up the full call.

The 3-part structure (applies to both modules)

[1] SERVER INSTRUCTIONS (system prompt) — YOU write this
Tells the LLM its role, what to ask, what NOT to do, and what format to reply in.

[2] WHAT PATIENT SAID — the running conversation so far
(transcript of patient's answers, fed in automatically)

[3] RELEVANT RAG CHUNKS — reference material retrieved from our knowledge base for this specific question/domain

Your job for each module is to design part (1), decide what goes into the knowledge base for part (3), and write the code that assembles all three into one LLM call.

Module A — Ayurveda (Prakriti + Vikriti)

These are two separate things that are easy to mix up. Prakriti is the patient's fixed constitutional type, assessed once. Vikriti is their current imbalance, assessed at every visit, and depends on the Prakriti baseline.

Step A1 — Prakriti: build the questionnaire (NOT an LLM call)

- This is a static, scored questionnaire — not a conversation. It runs once per patient, when they first register.
- Use the CCRAS Prakriti Assessment Scale as your question bank (the validated official scale — don't invent your own questions).
- Each question has fixed multiple-choice answers, each answer maps to Vata / Pitta / Kapha points.
- Output: a score like Vata 45%, Pitta 30%, Kapha 25% → store this against the patient's profile permanently.
- Deliverable: the question bank (digitized from the CCRAS scale) + a scoring function that turns answers into dosha percentages.

A note on scope: RAG is not your task

Someone else on the team owns building the symptom-to-dosha reference table and retrieving the right chunks (part 3). You don't need to build that table yourself. Your job is to write the system prompt (part 1)

so that it makes good use of whatever chunks get handed to it. Sync with the RAG person on what format those chunks arrive in (e.g. raw table rows vs. a pre-summarized paragraph) so your prompt expects the right shape of input.

Important: you'll receive TWO kinds of chunks, not one

RAG hands you chunks from two different sources for every question-turn — your prompt needs to treat them differently:

- **Clinical reference chunks** — general medical/Ayurvedic knowledge (symptom-to-dosha mappings, red-flag lists). Same for every patient. Use these to decide what's clinically relevant to ask next.
- **Patient history chunks** — THIS specific patient's own past records (e.g. a prior heart attack, a past visit note). Use these to raise urgency or context — e.g. chest pain in a patient with a documented prior MI should be treated with more urgency than chest pain alone, not just noted in passing.
- Handle the case where patient-history chunks are empty (new patient, no prior records) — your prompt should work fine without crashing or awkwardly referencing history that doesn't exist.

Step A2 — Write the Vikriti system prompt (this is part 1)

Write a system prompt that does the following. Use this as your template:

ROLE: "You are assessing dosha imbalance (Vikriti) for an Ayurvedic consultation. The patient's baseline Prakriti is {prakriti_result}."

TASK: "Ask ONE question at a time to identify symptoms indicating deviation from the patient's baseline. Use the retrieved reference table to map symptoms to dosha aggravation. Consider that a symptom may be NORMAL for this patient's Prakriti and not an imbalance — e.g. mild heat tolerance in a Pitta-dominant patient may be baseline, not aggravation."

CONSTRAINTS: "Do not diagnose a Western/allopathic disease. Do not repeat a question already answered. If patient history chunks show a relevant prior condition, factor that into your questioning and note it in your summary. Stop and summarize once you have high confidence, or after a reasonable number of questions."

OUTPUT FORMAT: reply ONLY in JSON:
EITHER {"next_question": "..."}
OR {"vikriti_summary": "...", "dosha_scores": {...},
"confidence": 0-1}

Test this by role-playing 4-5 patients yourself (typing patient answers manually) and checking the questions it asks make clinical sense before wiring it to real ASR input.

Step A3 — Wire it up

- On each turn: fetch patient's stored Prakriti result, take the RAG chunks handed to you by the RAG teammate, assemble [system prompt + Prakriti] + [conversation so far] + [retrieved chunks], call the LLM.
- Parse the JSON response — if next_question, show it to patient; if vikriti_summary, save it and end this section.

Module B — Allopathy (SOCRATES framework)

This mirrors Module A exactly — same 3-part structure, different content. SOCRATES is the standard framework doctors use to take a symptom history (taught in every MBBS curriculum, so reference material is easy to find).

Step B1 — Know the 8 axes you're collecting

- **Site** — where exactly is the symptom located
- **Onset** — sudden or gradual, when it started
- **Character** — sharp, dull, burning, throbbing, etc.
- **Radiation** — does it spread or move anywhere
- **Associated symptoms** — anything else alongside it
- **Time course** — constant, intermittent, changing over time
- **Exacerbating/relieving factors** — what makes it better or worse
- **Severity** — usually rated 1-10

Step B2 — Write the SOCRATES system prompt (part 1)

Same note as the Ayurveda module: you don't build the red-flag list or differential reference material yourself — the RAG teammate retrieves relevant chunks based on whatever the patient just said (e.g. patient says "my chest is paining" → RAG hands you chunks specifically about chest-pain red flags and which SOCRATES axes matter most for that symptom, not the whole reference library). Your prompt should be written to make good use of whatever targeted chunks show up, not assume a fixed table.

ROLE: "You are taking a structured symptom history using the SOCRATES framework."

TASK: "Ask ONE question at a time, covering: Site, Onset, Character, Radiation, Associated symptoms, Time course, Exacerbating/relieving factors, Severity – in whatever order fits the conversation. Do not ask about an axis already covered."

CONSTRAINTS: "Do not diagnose. Do not suggest medication. If the patient describes a red-flag symptom (see reference chunks), OR if patient history chunks show a relevant prior condition (e.g. a past heart attack alongside current chest pain), treat this as elevated urgency: stop the questionnaire and flag for immediate escalation instead of continuing."

OUTPUT FORMAT: reply ONLY in JSON:

EITHER {"next_question": "..."}
OR {"socrates_summary": {"site": "", "onset": "", "character": "",

"radiation": "", "associated": "", "timecourse": "",
"factors": "", "severity": ""}, "confidence": 0-1,
"red_flags": []}

Step B3 — Wire it up

Same integration pattern as Module A: assemble [system prompt] + [conversation so far] + [retrieved chunks handed to you by RAG], call the LLM, parse JSON, either continue asking or save the summary. If red_flags is non-empty, this should interrupt the normal flow.

Quick checklist before calling either module 'done'

- Prakriti: CCRAS question bank digitized + scoring function working, tested on a few sample answer sets.
- Vikriti: system prompt drafted, tested with 4-5 manually role-played patients (using placeholder/sample RAG chunks if the real RAG pipeline isn't ready yet).
- SOCRATES: system prompt drafted, tested with 4-5 manually role-played patients (same placeholder-chunk approach if needed).
- Both prompts: tested at least one scenario where patient-history chunks should change the outcome (e.g. chest pain + prior heart attack raising urgency) to confirm your prompt actually reacts to it, not just the general reference chunks.
- Both prompts: tested with empty patient-history chunks (new patient) to confirm nothing breaks.
- Synced with the RAG teammate on the exact format chunks will arrive in (raw rows vs. summarized text, and the split between clinical-reference vs. patient-history chunks) for both modules.
- Both modules: LLM always replies in the specified JSON — no free text that breaks parsing.
- Both modules: confirmed it never states a diagnosis or prescribes anything — only collects structured history.

If anything here is unclear, flag it early rather than guessing — this logic is the core differentiator of our submission, so it's worth getting right over getting it done fast.